Dataflow Systems

DSC 102: Systems for Scalable Analytics

Haojian Jin

UC San Diego

Where we are

- **L4:** split the **tasks** across workers
- **L5:** split the **data** across one machine's memory hierarchy
- **L6:** split the data across **machines**, and wrote every data-parallel operation by hand, BSP style
- **Today:** the systems that do that plumbing **for you**: MapReduce, Hadoop, Spark

Dataflow systems: why they exist, how MapReduce runs, what Spark adds, and how they compare with Dask.

Q: Who moves the pages?

In L5 and L6 *you* decided which pages go from disk to DRAM, which shards go to which node, and when the workers synchronize.

How do we shield users from thinking about moving raw pages between disk, DRAM, and the network when scaling data-intensive programs?

Relational databases answered this for SQL half a century ago (recall Codd in L4). Today's question is what happened when the workload stopped being SQL.

Four questions for today

1. Why go **beyond parallel RDBMSs** at all?
2. What is **MapReduce**, and how does it run?
3. What does **Spark** add with **dataflow programming**?
4. How do dataflow systems compare with **task-parallel** systems such as Dask?

Q1 · Parallel RDBMSs

Parallel RDBMSs are highly successful and widely used: typically **shared-nothing data parallelism**, exactly the L6 recipe.

Optimized runtime performance plus enterprise-grade features:

- ANSI SQL and more
- Business Intelligence (BI) dashboards and APIs
- Transaction management and crash recovery
- Indexes, auto-tuning, and so on

Q: So why did people need to go beyond parallel RDBMSs?

Take CSE 132C for more on parallel RDBMSs.

Beyond RDBMSs: a brief history

The database community got **blindsided** by the rise of the Web and Internet giants (Google, Amazon, Yahoo, Facebook).

Four new concerns of the Web giants, versus RDBMSs built for enterprises:

Concern	What the giants needed
Developability	Custom data models and computations that were hard to program in SQL; simpler APIs
Fault tolerance	Scale to **thousands** of machines; graceful handling of worker failure
Elasticity	Easily upsize or downsize the cluster with the workload
Cost	Commercial RDBMS licenses were too costly; they hired engineers to build their own systems

A new breed of parallel data systems, dataflow systems, jolted the database world out of its complacency.

Q2 · What is MapReduce?

- A **programming model** for parallel programs on sharded data, plus a **distributed system architecture** that runs them
- *Map* and *Reduce* are terms from functional programming; the engineer implements the logic of Map and Reduce
- The system handles data distribution, parallelization, and fault tolerance under the hood
- Created by **Google** to solve a "simple" workload: index, store, and search the Web
- Google's engineers started with **MySQL** and abandoned it for the reasons on the previous slide

Dean and Ghemawat, MapReduce: Simplified Data Processing on Large Clusters, OSDI 2004.

Programming-language background

Paradigm	Idea	Example
Declarative	Say *what* result you want, not how to get it	SQL
Imperative	The micromanaging boss: instructions down to the last detail	Python
Object-oriented	Organize data and code around classes and objects	Java
Functional	Build programs from expressions and functions, **without mutating state**	Lisp, Haskell

Map and Reduce come from the functional world: pure functions over records, so the system is free to run them anywhere, in any order, and to re-run them after a failure.

The standard example: word count

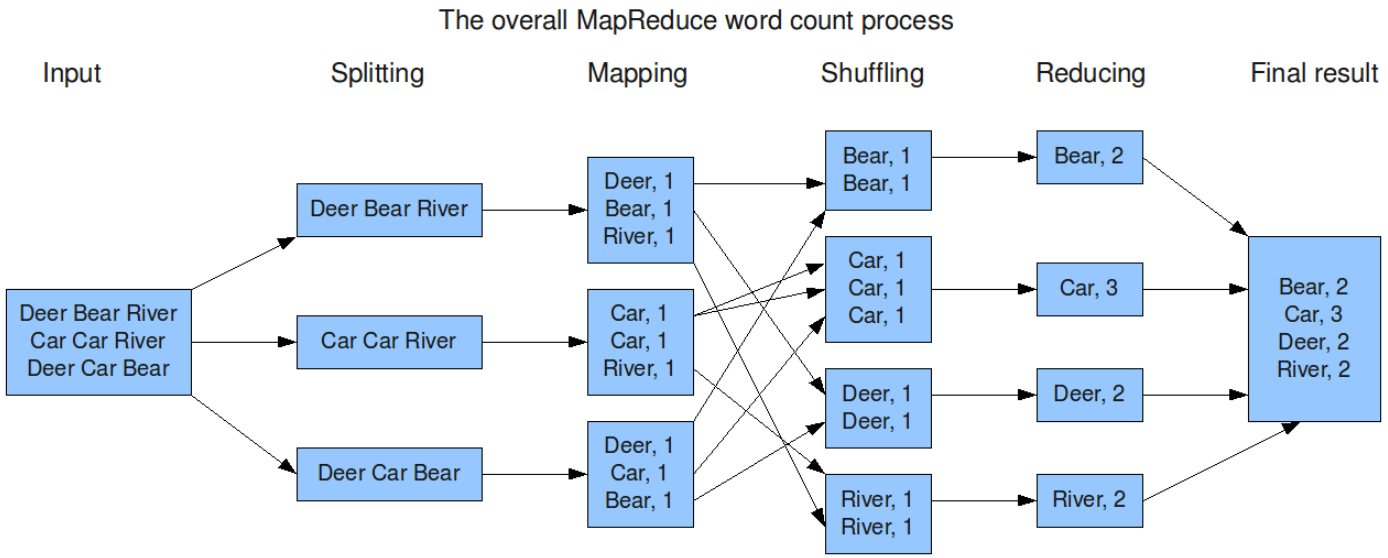
```
function map(String docname, String doctext):
    for each word w in doctext:
        emit (w, 1)

function reduce(String word, Iterator partialCounts):
    sum = 0
    for each pc in partialCounts:
        sum += pc
    emit (word, sum)
```

- **Input:** a set of text documents (say, webpages)
- **Output:** a dictionary of unique words and their counts
- `emit` is part of the MapReduce API
- The engineer writes only these two functions

Hmm, sounds suspiciously familiar: this is a GROUP BY with a COUNT.

How MapReduce works



- Each Mapper and Reducer is a separate **process**; Reducers wait at a **barrier** (BSP from L6)
- Fault tolerance: the input is **replicated** (HDFS); a failed Mapper or Reducer is simply **re-run**

Abstract semantics of MapReduce

Map(): process one "record" at a time, independently

- A record can physically batch multiple data examples or tuples
- Dependencies across Mappers are **not allowed**
- Emit one or more key-value pairs; input and output types may differ

Reduce(): gather all Map outputs across workers that share the same key into an **Iterator**

- Apply the aggregation function to the Iterator to get the final output(s)

Input split: the physical-level shard that batches many records into one file block

- HDFS default block size: 128 MB; applications can define custom splits

Benefits and the catch of MapReduce

Goal: high-level functional operations that simplify data-intensive programs.

Key benefits

- Map() and Reduce() are **highly general**: any data types or structures; great for ETL, text, multimedia
- Native scalability and large-cluster parallelism
- The system handles **fault tolerance** automatically
- Open-source stacks: Hadoop, later Spark

The catch

- Users must learn the "art" of casting a program as MapReduce
- Map operates **record-wise**; Reduce aggregates **globally** by key
- Libraries exist in C/C++, Java, Python, R, Scala

Emulate MapReduce in SQL?

Q: How would you count words in an RDBMS?

Step 1: transform the documents into a relation and load it (the ETL stage). Pre-split each document into words:

```
DocWords(DocName, Word)
```

Step 2: a single, simple SQL query

```
SELECT   Word, COUNT(*)
FROM     DocWords
GROUP BY Word
[ORDER BY Word]
```

Parallelism, scaling, and fault tolerance are done by the RDBMS under the hood. MapReduce exposes the same two phases to the programmer.

The L6 operations, written as MapReduce jobs

Operation	Map() on each tuple	Reduce()
Select	Apply the condition; if it holds, emit (dummy key, tuple)	**Not needed**: a "map-only" job
Simple algebraic aggregate (SUM, AVG, MAX)	Compute incremental stats; emit (single global dummy key, stats)	One key, so the Iterator holds all sufficient stats; unify into the global aggregate
GROUP BY aggregate	Compute incremental stats; emit (**grouping attribute**, stats)	Each Iterator holds one group's stats; different Reducers output different groups
Matrix norm (an algebraic aggregate)	Compute incremental stats per tile; emit (dummy key, stats)	Unify, exactly like a simple aggregate

Input split in every case: shard the table tuple-wise. The three L6 ingredients again; the system writes the loop.

Batch gradient descent in MapReduce

$$\nabla L(\mathbf{w}^{(k)}) = \sum_{i=1}^{n} \nabla l\big(y_i, f(\mathbf{w}^{(k)}, x_i)\big)$$

- An algebraic aggregate with **vector addition** as the aggregation
- **Input split:** shard the table tuple-wise; broadcast the current $\mathbf{w}^{(k)}$ to every Mapper
- **Map():** on each tuple compute the per-example gradient; add these across the examples in the shard; emit the partial sum under a single dummy key
- **Reduce():** one global key, so the Iterator holds all partial gradients; add them to get the full batch gradient

  One job per epoch; the driver updates $\mathbf{w}$ and launches the next. The L5 BGD access pattern, with none of the plumbing.

What is Hadoop, then?

- The **free and open-source** implementation with **MapReduce** as the programming model and **HDFS** as the filesystem (recall HDFS from L6)
- User writes Map and Reduce; input splits, data distribution, shuffling, and fault tolerance are handled by Hadoop under the hood
- Exploded in popularity in the 2010s: hundreds of papers, dozens of products; a "revolution" that took the database world by surprise
- Nowadays the Hadoop MapReduce **API is largely supplanted by Spark**; HDFS is still common in many companies

Do not confuse MapReduce (the model) with Hadoop (one implementation), or vice versa.

Q3 · Apache Spark

- **Dataflow programming model** that subsumes most of relational algebra and MapReduce
- Inspired by the pandas style of **chaining functions**
- Unified storage of relations, text, and more; custom programs
- System implementation (re)designed from scratch; tons of sponsors, enormous hype
- **Key idea vs. Hadoop:** exploit **distributed memory** to cache data between steps
- **Key novelty vs. Hadoop: lineage-based** fault tolerance
- Open-sourced to Apache; commercialized as **Databricks**

Zaharia et al., Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing, NSDI 2012.

Why Spark, in Zaharia's words

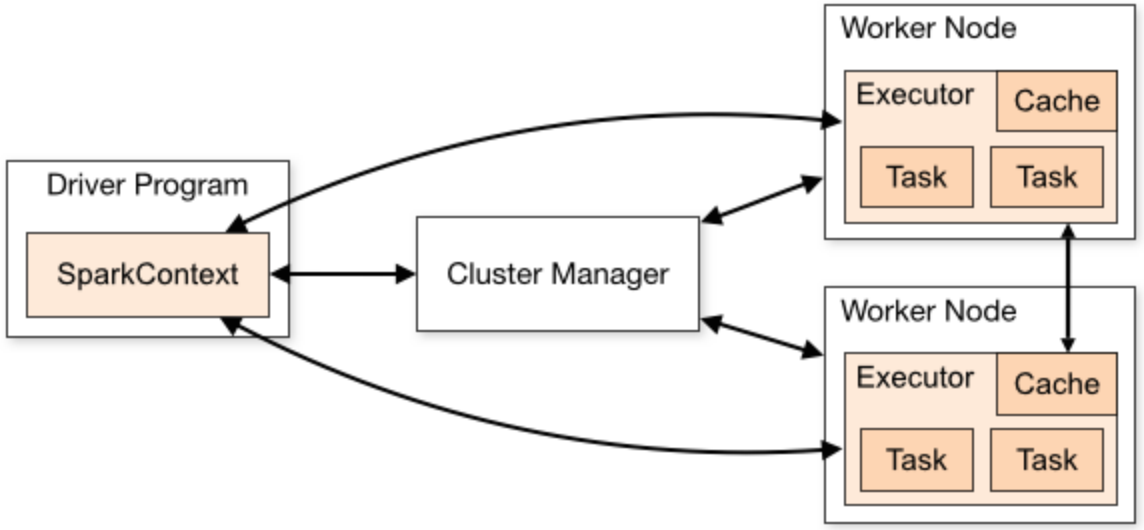
> "Google's MapReduce presented a simple and general model for batch processing that automatically handles faults. However, MapReduce was found poorly suited for other types of workloads, leading to a wide range of *specialized* models ... Pregel, F1, MillWheel, Storm, Impala, Piccolo, GraphLab ... it seems that cluster computing is bound to require an array of point solutions for different tasks."

> "We show that a simple extension to MapReduce called **Resilient Distributed Datasets (RDDs)**, which just adds efficient **data sharing** primitives, greatly increases its generality."

> "RDDs would be less suitable for applications that make **asynchronous fine-grained updates to shared state**, such as a storage system for a web application or an incremental web crawler."

Zaharia, An Architecture for Fast and General Data Processing on Large Clusters, UC Berkeley Tech Report EECS-2014-12.

Spark's distributed architecture



- The **driver program** holds the `SparkContext` and the dataflow graph
- A **cluster manager** (standalone, YARN, Kubernetes) hands out resources
- Each worker node runs an **executor** with tasks and a **cache**: the distributed memory that Hadoop lacked

The driver and manager roles are the L6 manager, split in two.

spark.apache.org/docs/latest/cluster-overview.html

Spark's dataflow programming model

Transformations	*map*(*f* : T ⇒ U)	:	RDD[T] ⇒ RDD[U]
	filter(*f* : T ⇒ Bool)	:	RDD[T] ⇒ RDD[T]
	flatMap(*f* : T ⇒ Seq[U])	:	RDD[T] ⇒ RDD[U]
	sample(*fraction* : Float)	:	RDD[T] ⇒ RDD[T] (Deterministic sampling)
	groupByKey()	:	RDD[(K, V)] ⇒ RDD[(K, Seq[V])]
	reduceByKey(*f* : (V, V) ⇒ V)	:	RDD[(K, V)] ⇒ RDD[(K, V)]
	union()	:	(RDD[T], RDD[T]) ⇒ RDD[T]
	join()	:	(RDD[(K, V)], RDD[(K, W)]) ⇒ RDD[(K, (V, W))]
	cogroup()	:	(RDD[(K, V)], RDD[(K, W)]) ⇒ RDD[(K, (Seq[V], Seq[W]))]
	crossProduct()	:	(RDD[T], RDD[U]) ⇒ RDD[(T, U)]
	mapValues(*f* : V ⇒ W)	:	RDD[(K, V)] ⇒ RDD[(K, W)] (Preserves partitioning)
	sort(*c* : Comparator[K])	:	RDD[(K, V)] ⇒ RDD[(K, V)]
	partitionBy(*p* : Partitioner[K])	:	RDD[(K, V)] ⇒ RDD[(K, V)]
Actions	*count*()	:	RDD[T] ⇒ Long
	collect()	:	RDD[T] ⇒ Seq[T]
	reduce(*f* : (T, T) ⇒ T)	:	RDD[T] ⇒ T
	lookup(*k* : K)	:	RDD[(K, V)] ⇒ Seq[V] (On hash/range partitioned RDDs)
	save(*path* : String)	:	Outputs RDD to a storage system, *e.g.*, HDFS

- **Transformations** are relational operators, MapReduce, and friends, as functions from RDD to RDD; they are **lazy**
- **Actions** are what force computation (recall Dask's `compute()` in L4)

Zaharia et al., NSDI 2012, Table 2.

Word count in Spark

RDD API (Python; also Scala, Java, R)

```
text_file = sc.textFile("hdfs://...")
counts = text_file.flatMap(lambda line: line.split(" ")) \
                  .map(lambda word: (word, 1)) \
                  .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs://...")
```

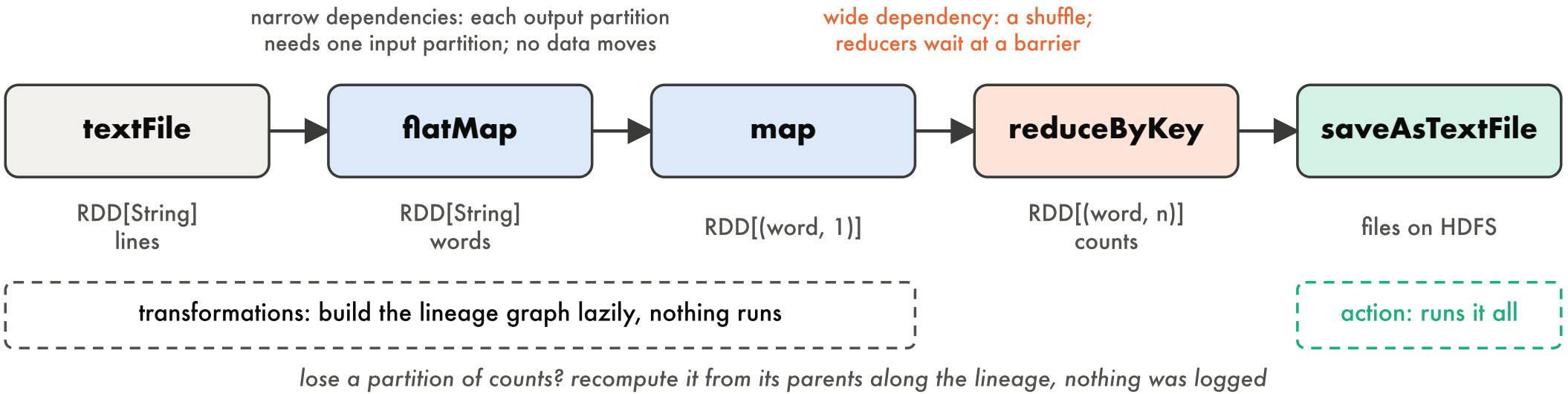
DataFrame API of Spark SQL

```
words = spark.read.text("hdfs://...") \
    .select(explode(split("value", " ")).alias("word"))
counts = words.groupBy("word").count()
counts.write.csv("hdfs://...")
```

Or plain SQL on the same table. The two styles **interleave** freely.

Same two functions as the MapReduce version, now as a chain of calls. Nothing runs until the save.

Lineage: how Spark remembers and recovers



The dataflow graph is the recovery log. Caching an RDD in memory is an optimization on top; the lineage is the guarantee.

Spark DataFrame API and Spark SQL

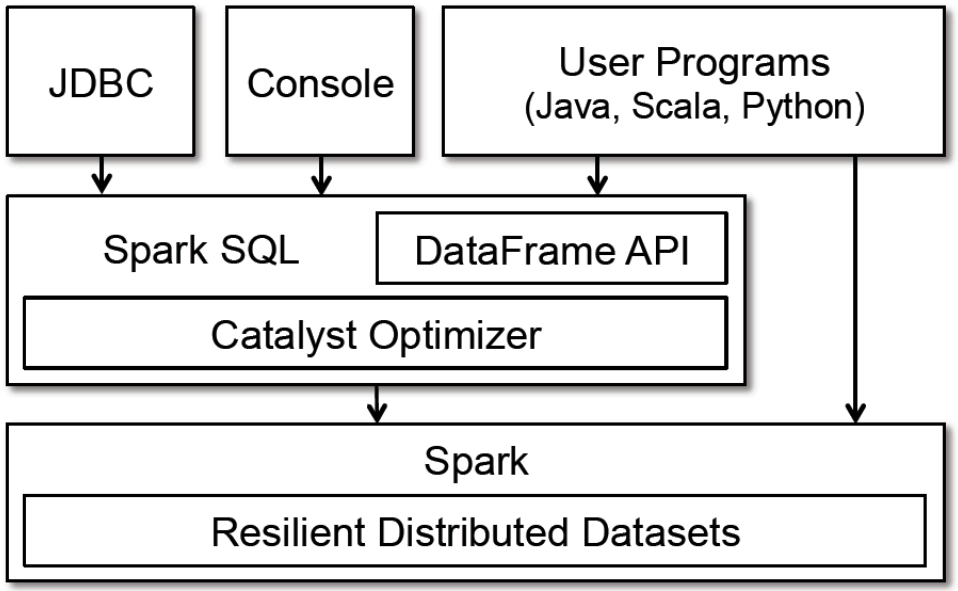


Figure 1: Interfaces to Spark SQL, and interaction with Spark.

- Databricks now recommends **Spark SQL and the DataFrame API**; avoid the RDD API unless really needed
- **Key reason:** automatic **query optimization** becomes feasible once the system knows the columns and the operators
- Also known as (painfully) re-learning **forty years** of database systems research

Armbrust et al., Spark SQL: Relational Data Processing in Spark, SIGMOD 2015, Figure 1.

Query optimization in Spark

Common automatic query optimizations from the RDBMS world are now performed in Spark's **Catalyst** optimizer:

- **Projection pushdown:** drop unneeded columns early on
- **Selection pushdown:** apply predicates close to the base tables
- **Join order optimization:** not all join orders are equally costly
- **Fusing of aggregates**, and more

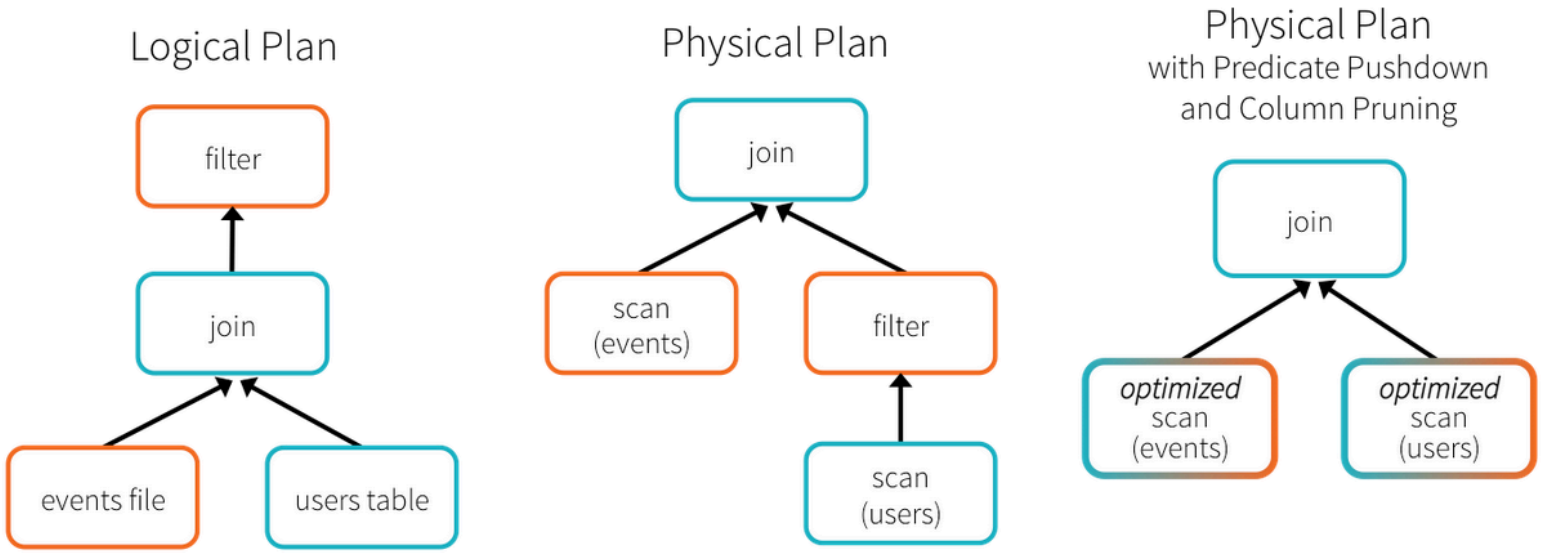
Every one of these is an I/O argument from L5: read fewer bytes, read them once, keep the intermediate data small.

Armbrust et al., SIGMOD 2015. Take CSE 132C for more on relational query optimization.

Catalyst on an example

```
def add_demographics(events):
    u = sqlCtx.table("users")                    # Load partitioned Hive table
    events \
      .join(u, events.user_id == u.user_id) \    # Join on user_id
      .withColumn("city", zipToCity(df.zip))     # Run udf to add city column

events = add_demographics(sqlCtx.load("/data/events", "parquet"))
training_data = events.where(events.city == "New York").select(events.timestamp).collect()
```



Databricks is building yet another parallel RDBMS.

Armbrust et al., Spark SQL: Relational Data Processing in Spark, SIGMOD 2015.

Reinventing the wheel?



Each generation rebuilt the wheel for a new vehicle.

Dataflow systems rebuilt the parallel RDBMS for a new workload: unstructured data, custom code, thousands of cheap machines, and open source.

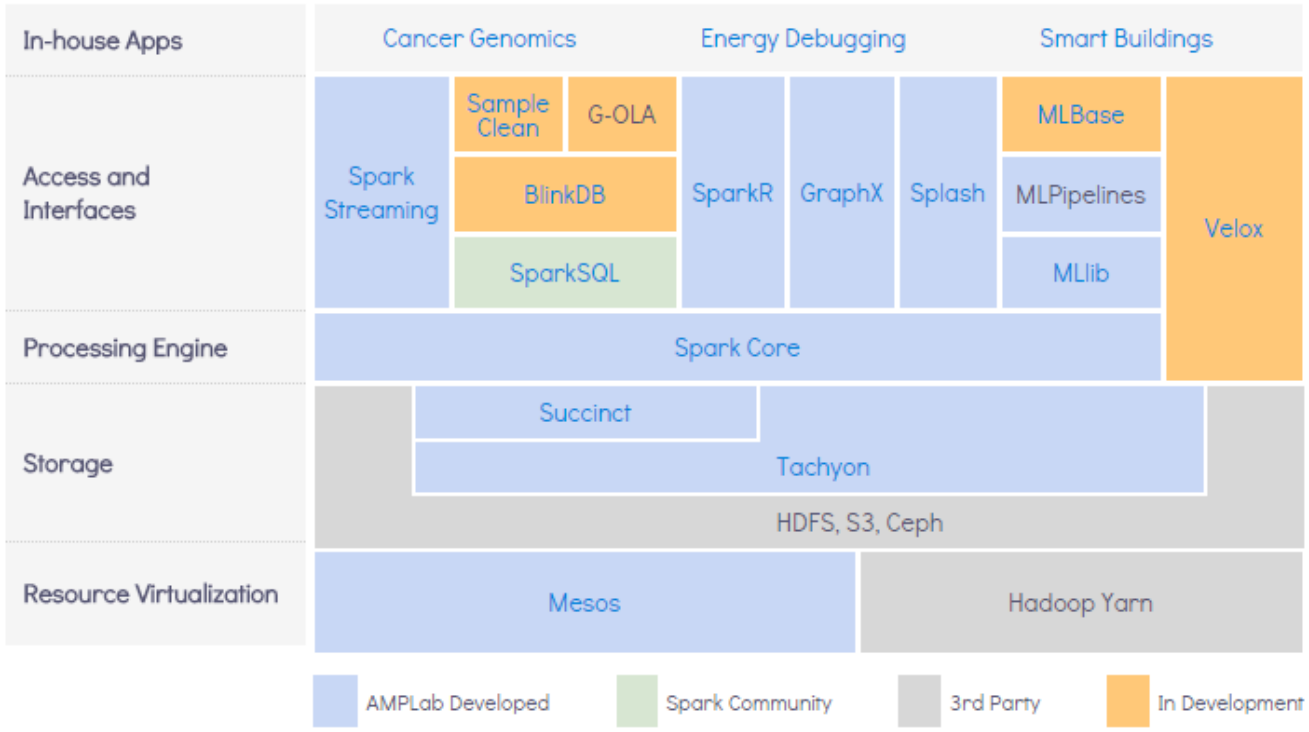
The ideas survived; the packaging changed.

Comparing Spark's APIs

	RDD	DataFrame	pandas API on Spark (formerly Koalas)
Abstraction level	Low	High	High
Named columns	No	Yes	Yes
Query optimization	No	Yes	Yes
Programming mode	map-reduce	Dataflow, SQL	pandas-like
Best suited for	Unstructured data, low-level operations, fans of functional programming	Structured data, high-level operations, people who know SQL, Python, R	Structured data, lowest barrier for people who only know pandas or Dask

Take DSC 291 (Spark) to learn more Spark programming.

The Spark-based ecosystem of tools



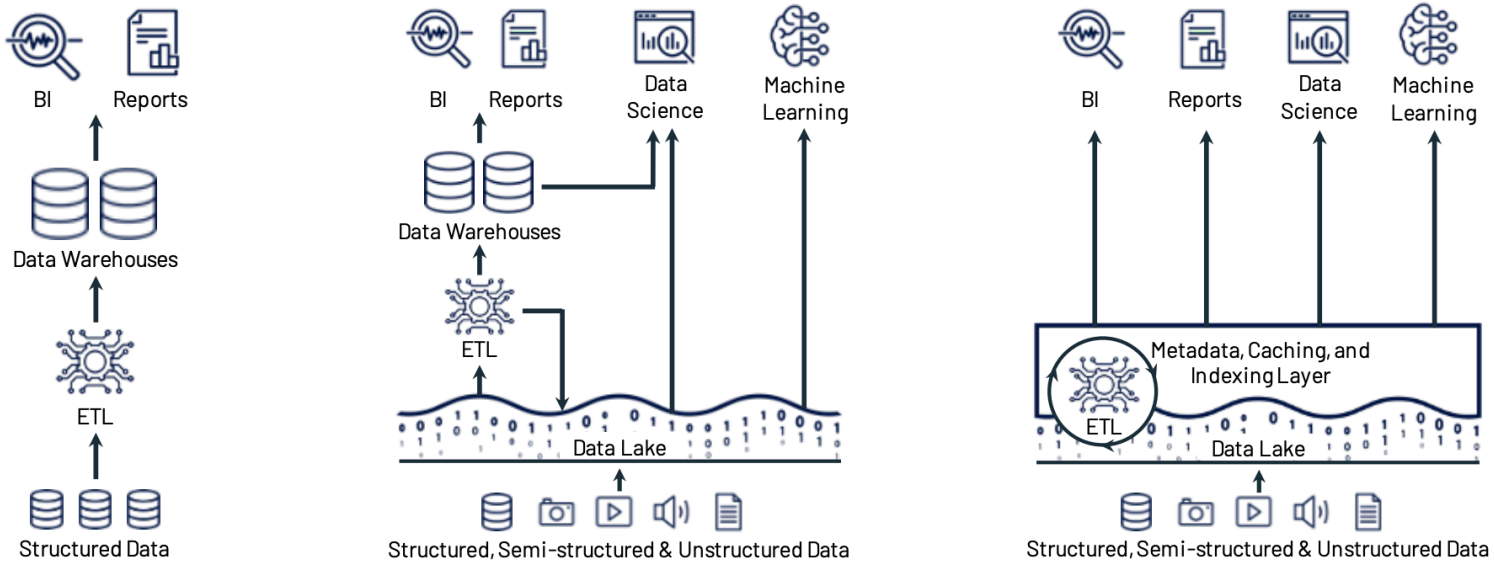
The **Berkeley Data Analytics Stack (BDAS)**, circa 2015:

- Resource virtualization: Mesos, YARN
- Storage: HDFS, S3, Tachyon
- Processing engine: **Spark Core**
- Interfaces: Spark SQL, Streaming, MLlib, GraphX, SparkR

Most of the upper layers still exist inside Spark today.

amplab.cs.berkeley.edu/software (2015).

The new paradigm: the data "lakehouse"



(a) First-generation platforms. (b) Current two-tier architectures. (c) Lakehouse platforms.

Figure 1: Evolution of data platform architectures to today's two-tier model (a-b) and the new Lakehouse model (c).

A data **lake** decouples the file format from the processing stack and serves many frontends; a **lakehouse** adds a metadata, caching, and indexing layer on top.

Q4 · Dataflow systems versus task-parallel systems

Candidate arguments; we will build the list together in class.

	Dataflow systems (MapReduce, Spark)	Task-parallel systems (Dask, L4)
Data	**Partitioned**: each worker reads its shard	**Replicated** to every worker that needs it
Parallelism	Automatic, from the operators	From the task graph you (or the library) build
Fault tolerance	Built in: replication and lineage	Weaker; often restart the task
Optimization	Query optimizer over a restricted set of operators	Little; you tune chunk sizes and task counts
Flexibility	Must fit the operators; shuffles are costly	Arbitrary Python functions as tasks
Idle workers	Barriers and stragglers	Falling degree of parallelism

The pros of one are the cons of the other.

More specific to Spark versus Dask

Discussion in class. Some threads to pull on:

- **Runtime:** JVM with Python bindings (serialization across the boundary) versus native Python
- **Semantics:** Spark DataFrames are relational and unordered; Dask DataFrames try to be pandas
- **Scale:** Spark runs on thousands of nodes in production; Dask shines from a laptop to a mid-size cluster
- **Workloads:** SQL, ETL, streaming versus array computing, custom task graphs, scientific Python
- **Operations:** a cluster manager and a JVM to run, versus `pip install`

Q: For PA1 and PA2, which of these did you feel?

Review questions

1. Name the four concerns that drove the Web giants beyond parallel RDBMSs.
2. Write Map() and Reduce() for a GROUP BY SUM. Which operation needs no Reduce() at all?
3. What is a *shuffle*, and which L6 concept does the Reducer barrier correspond to?
4. What is the difference between MapReduce and Hadoop?
5. What does lineage buy Spark that Hadoop had to pay for with disk writes?
6. Why does Databricks recommend the DataFrame API over the RDD API?
7. Name one pro and one con of a dataflow system compared with a task-parallel system.

Key takeaways

1. **Dataflow systems automate the plumbing of L6:** sharding, node-local work, the barrier, the unification, and recovery
2. **MapReduce** is a programming model, two pure functions over records; **Hadoop** is one implementation of it
3. **Spark** adds in-memory caching between steps, **lineage** instead of logging, and a dataflow API that subsumes relational algebra and MapReduce
4. The **DataFrame API** brought the query optimizer back, and with it forty years of database ideas
5. Every generation rebuilds the parallel RDBMS for its workload; knowing the ideas lets you see through the packaging

Optional · Primer: K-means clustering

Not part of the syllabus: an advanced example of scaling an ML algorithm with MapReduce.

Basic idea: identify clusters based on Euclidean distances; formulated as an optimization problem. **Lloyd's algorithm** is the most popular heuristic:

- **Input:** n × d examples (points). **Output:** k clusters and their centroids

1. Initialize k centroid vectors and a point-to-cluster assignment
2. **Assignment step:** scan the dataset and assign each point to the nearest centroid
3. **Update step:** given the new assignment, scan the dataset again to recompute every centroid
4. Repeat 2 and 3 until convergence or a fixed number of iterations

Optional · K-means in MapReduce

Shard tuple-wise; each point has an ExampleID. **Two jobs per iteration**, with two small matrices broadcast through HDFS: the dense k × d centroids **A** and the sparse n × k assignments **B**.

Job 1: assignment (reads A, writes B)

- **Map():** read A; find the nearest of the k centroids; emit (PointID, ClusterID)
- **No Reduce():** map-only; the new B lands on HDFS

Job 2: update (reads B, writes A)

- **Map():** read B; look up the point's cluster; emit (ClusterID, point)
- **Reduce():** all points of one cluster; average them; emit (ClusterID, centroid)

Each iteration is a map-only job plus a GROUP BY AVG.